

UML PROYECTO PROGRAMACIÓN

David Serrano Molina
Thiago Matias Kojakovic Do Santos

Índice

Descripción del Sistema	3
Estructura del Sistema	3
Diagramas	4
Diagrama de clases	4
Diagrama de secuencia	4
Inicio de sesión	4
Dar de alta pregunta	5
Diagrama de casos de uso	5
Justificación del diseño	6

Descripción del Sistema

El sistema es una aplicación de escritorio que gestiona un banco de preguntas que luego utiliza para generar exámenes. Ese banco de preguntas no es inmutable, se pueden editar o borrar preguntas existentes así como también pueden crear nuevas preguntas.

Por último, no cualquiera debe poder acceder al sistema así que la aplicación cuenta con un sistema de inicio de sesión para probar que quien accede son usuarios verificados.

Para almacenar toda esta información el sistema accede a una base de datos que almacena tanto los usuarios con sus contraseña cifradas como las preguntas junto con sus respuestas.

Estructura del Sistema

La estructura de la aplicación está basada en el modelo vista-controlador.

Este modelo conlleva separar las funciones del programa en clases individuales haciendo más claro y fácil de mantener el código.

Todas las vistas del programa están almacenadas en una clase (Vista_Principal) y a la hora de necesitar cambiar de vista se hace uso del método `show_view`. Para registrar el evento de cada vista se cuenta con varios controladores que los capturan y funcionan de intermediarios entre el modelo y las vistas.

El modelo consta de varias clases de utilidad que almacenan en varias colecciones los objetos que representan cada registro de la base de datos. También almacena los cambios producidos en la sesión para posteriormente aplicarlos a la base de datos.

Es importante recalcar que se almacena información en el programa pero solo en tiempo de ejecución. La persistencia de los cambios realizados es gracias a la base de datos.

Diagramas

Diagrama de clases

El diagrama de clases ofrece una vista de todas las clases que componen el programa junto con sus atributos y sus métodos. También muestran las relaciones que tienen entre sí, sus cardinalidades y en nuestro caso el cómo se organizan para que no incumplan el modelo vista-controlador.

Para respetar el modelo, se tuvo en cuenta en el desarrollo de las clases pero sobre todo al establecer las relaciones ya que una vista solo puede tener relación con su controlador correspondiente y con la clase principal (es una relación de dependencia ya que forma parte como atributo).

En cuanto a la cardinalidad de las relaciones, la mayoría de casos eran de 1 a 1 así que para evitar ser redundante y confundir al ser interpretado sólo han sido marcadas las relaciones diferentes a 1:1.

Diagrama de secuencia

En el diagrama de secuencia se representa la interacción entre las diferentes clases y componentes del programa en una situación en específico. Siempre empieza con una solicitud por parte del usuario y siempre acaba con una respuesta por parte del sistema a esta solicitud. En nuestro caso desarrollamos los siguientes casos.

Inicio de sesión

El proceso es el siguiente:

1. El usuario introduce las credenciales de usuario en la vista VW_Login y presiona el botón “aceptar”.
2. La VW_Login produce un evento que Controlador_Login gestiona.
3. Controlador_Login cifra la contraseña y abre una conexión SQL.
4. Controlador_Login comprueba si la combinación usuario-contraseña y cierra la conexión.
5. Si la comprobación es correcta entonces utiliza Vista_Principal para cambiar a VW_Main_Menu y crea un objeto Usuario para guardar el usuario en uso.

Dar de alta pregunta

El proceso es el siguiente

(Desde VW_Main_Menu)

1. El usuario presiona el botón “Preguntas”.
2. VW_Main_Menu produce un evento que Controlador_Main_Menu gestiona.
3. Controlador Main_Menu abre una conexión y obtiene el listado de todas las preguntas y respuestas actuales y las guarda en un objeto DB_Preguntas.
4. Controlador Main_Menu carga un listado de J_Records usando el objeto de DB_Preguntas en VW_Preguntas e indica a la Vista_Principal que cambia a VW_Preguntas.
5. El usuario presiona el botón “Nuevo”.
6. VW_Preguntas crea un evento que Controlador_Preguntas gestiona.
7. Controlador_Preguntas indica a Vista_Principal que cambie a VW_Editar_Preguntas.
8. El usuario rellena los campos del formulario y presiona el botón guardar produciendo un evento.
9. Controlador_Preguntas gestiona el evento creando un objeto con los datos de la pregunta y abre una conexión SQL para actualizar la base de datos.
10. Si es un éxito devuelve un true ocasionando que Controlador_Preguntas le indique a Vista_Principal que cambie la vista a VW_Preguntas.

Diagrama de casos de uso

Básicamente el diagrama muestra cómo se mueve todo el sistema de preguntas y respuestas pero visto desde lo que cada uno puede hacer.

El usuario es el que más interactúa: inicia sesión, consulta preguntas, ve respuestas, actualiza sus datos y responde. Cada vez que hace algo importante, el sistema primero valida o se conecta a la base de datos para poder traer o guardar información. El administrador ya entra más en modo control, porque puede registrar y modificar preguntas.

Y algo clave es que casi todo deja rastro en los logs, o sea, el sistema guarda lo que pasa internamente. En general, el diagrama deja súper claro quién hace qué y cómo algunas acciones dependen de otras, sin meterse todavía en código pesado.

Justificación del diseño

A lo largo de la planificación del diseño hubo que tomar las siguientes decisiones:

- Usar el MVC suponía añadir un grado de dificultad al desarrollo del proyecto pero se aplicó porque facilita el mantenimiento del código, la escalabilidad del sistema y permite modificaciones aisladas sin afectar al resto del sistema (añadir clases al modelo o editar vistas sin afectar al funcionamiento del programa).
- En las fases tempranas la intención era usar un solo controlador pero esto ocasiona que con el tiempo fuese gigante haciendo imposible aplicar cualquier cambio o crear nuevas funciones. Al separarlos en varios controladores se provoca una mejor claridad en la lógica y modularidad en los controladores.
- Uso de clases intermedias como Logs o DB_preguntas se decidió para reducir el número de conexiones al servidor de la aplicación (el tráfico en la red) y por motivos de seguridad para evitar la interceptación de los paquetes si estos son constantes.
- Vista_Principal, los motivos son simples al utilizar una clase que contenga todas las vistas. Evitar dependencias entre vistas, ganas modularidad si se incluyen futuras vistas y centralizar la navegación mediante el método Show_Vista.
- El motivo de la clase Logs es mejorar la trazabilidad de las operaciones realizadas ya que los cambios realizados se guardan en archivo log mediante uno de los métodos de Logs.